Software Metrics and Their Use

Metrics are decision-support instruments, not truth

Davide Falessi

Agenda

- ISO/IEC 25010 quality models
- OO metrics (CK + selected design metrics)
- Function Points (functional size counting and estimation use)
- GQM+Strategies (linking goals, strategies, and measurement)

LEARNING OBJECTIVES

- Distinguish quality models (what "quality" means) from metrics (how attributes are operationalized and measured).
- Introduce ISO/IEC 25010 quality characteristics and explain how they support requirements specification and evaluation.
- Interpret OO design metrics (e.g., CK suite) as risk indicators and derive plausible refactoring/architecture actions.
- Perform a basic Function Point count (EI/EO/EQ/ILF/EIF), compute the weighted total (treat as FP unless an adjustment step is explicitly taught), and explain how FP is used for effort/cost estimation via calibrated productivity.
- Construct a minimal GQM+Strategies specification linking business goals → software goals → questions → measures → strategies/targets.

Measurement-to-decision pipeline

- Goal / decision (what decision is being supported?)
- Operational definition (what exactly is measured, on which artifact, under what rules?)
- Collection (tooling + frequency + cost)
- Analysis (baseline, trend, statistical test / control chart, comparisons)
- Interpretation rule (thresholds; what constitutes improvement?)
- Action (refactor, redesign, staffing, testing strategy, release decision)

ISO/IEC 25010:2011

System and software quality models

Quality model ≠ metrics: model defines *what* quality means; metrics define *how* to quantify it.

ISO STANDARD

- What ISO 25010 provides: a taxonomy of quality attributes (product quality + quality-in-use).
- What it does not provide: a complete measurement program; you must operationalize measures.
- How it is used in practice: requirement specification, QA planning, acceptance criteria, benchmarking.

ISO standard: Quality characteristics (Examples)

- **Functional suitability**
  - Functional completeness
  - Functional correctness
  - Functional appropriateness
- **Performance efficiency**
  - Time behaviour
  - Resource utilization
  - Capacity
- **Security**
  - Confidentiality
  - Integrity
  - Non-repudiation
  - Accountability
  - Authenticity
- **Maintainability**
  - Modularity
  - Reusability
  - Analysability
  - Modifiability
  - Testability

ISO STANDARD: BENEFITS

- Standardized vocabulary reduces ambiguity in quality requirements and trade-offs.
- Enables traceability from stakeholder needs → quality attributes → measurable acceptance criteria.
- If the business goal is ‘reduce churn’, ISO 25010 points to usability + reliability; you then define measurable indicators.

OO Metrics

OO METRICS

- Object-oriented metrics are used to measure the quality of object-oriented software systems.
- They help developers to identify design flaws, performance issues, and maintainability problems early in the development cycle.
- There are several object-oriented metrics, such as CK metrics, which can be used to evaluate the quality of an object-oriented software system.

OO Metrics: Interpretation Principle

- OO metrics are risk indicators; interpretation is **comparative** (baseline/trend) not absolute.

- Prefer distributions (top 10% worst classes) rather than global averages.

CK METRICS

- CK metrics, also known as Chidamber and Kemerer metrics, were developed by Shyam R. Chidamber and Chris F. Kemerer in 1994.

- CK metrics consist of six different metrics that can be used to measure various aspects of object-oriented software design, such as inheritance, coupling, and complexity.

CK METRICS

WMC (Weighted Methods per Class)

- Definition: sum of method complexities in the class (often approximated by method count).
- Interpretation: high WMC suggests large responsibility surface and higher change/test effort.
- Typical action: split class by responsibility; reduce per-method complexity.

DIT (Depth of Inheritance Tree)

- Definition: path length from the class to the root.
- Interpretation: high DIT can increase reuse but raises comprehension and override-interaction risk.
- Typical action: simplify inheritance; prefer composition where polymorphism is not essential.

NOC (Number of Children)

- Definition: number of immediate subclasses.
- Interpretation: high NOC may indicate reuse via inheritance; also increases regression risk when base changes.
- Typical action: stabilize base-class contracts; add targeted regression tests; reconsider hierarchy.

CK METRICS

CBO (Coupling Between Object classes)

- Definition: number of distinct classes to which the class is coupled (depends on tool/counting rule).
- Interpretation: high CBO suggests ripple effects and reduced modularity.
- Typical action: reduce dependencies; introduce interfaces/facades; isolate external services.

RFC (Response For a Class)

- Definition: number of methods potentially executed in response to a message (local + invoked methods).
- Interpretation: high RFC implies complex response behavior and larger test surface.
- Typical action: reduce call chains; simplify public API; extract collaborators.

LCOM (Lack of Cohesion in Methods)

- Definition: cohesion proxy; higher values typically mean methods operate on disjoint attribute sets.
- Interpretation: high LCOM suggests “multiple classes in one” (SRP violation) and harder maintainability.
- Typical action: refactor into cohesive classes/modules; relocate methods to the data they use.

Further OO metrics

In addition to CK metrics, many other object-oriented metrics can be used to measure various aspects of software design.

Some additional object-oriented metrics include:

- Method-level: CC, nesting depth
- Class-level: LOC/class, WMC, LCOM, RFC, CBO
- Package/module-level: afferent/efferent coupling, instability
- Process-level (often more predictive): code churn, change frequency, ownership concentration

Cyclomatic Complexity (CC)

- CC measures the complexity of a program by counting the number of independent paths through the code.
- Programs with high CC are more difficult to understand and maintain.
- CC can be calculated using tools such as static code analysis software.
- Use: testing (minimum #test cases for basis path), maintainability risk
- Reminder: thresholds are context dependent; prefer outliers and trends.

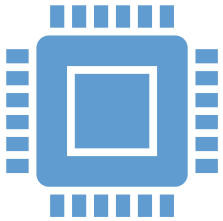
Fan-In and Fan-Out

- Fan-In measures the number of other classes that depend on a given class.
  - High fan-in → critical utility module; needs stability and strong tests

- Fan-Out measures the number of other classes that a given class depends on.
  - High fan-out → overly dependent class; refactor, reduce responsibilities

Function Points

FP: Definition



Function points are a software size measurement technique that quantifies the functionality provided by a software application



They are independent of programming language, technology, and implementation details



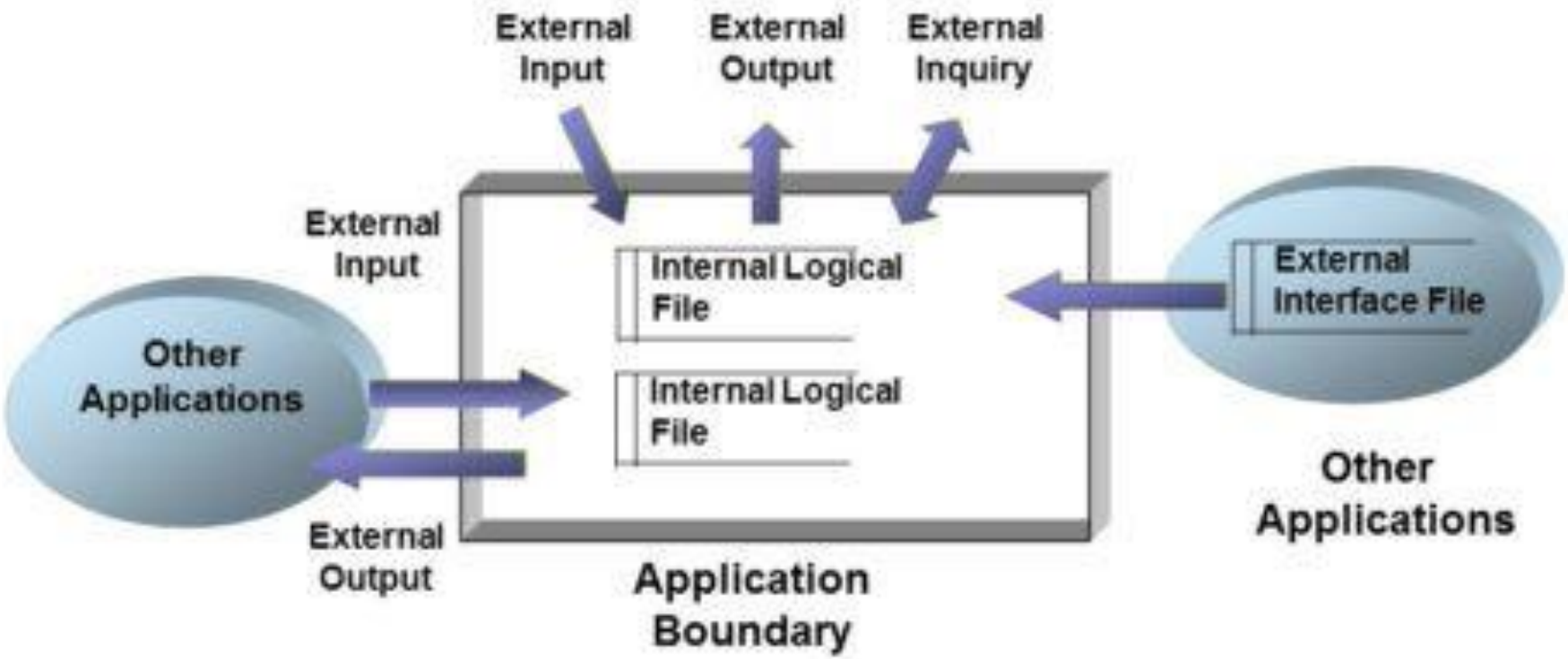
They are based on the user's perspective and the business value of the software application

FP: Use

- Primary use: early size estimation for cost/schedule (esp. outsourcing/contracts).

- Not for: not a direct measure of quality.

The Logical View of Function Point Analysis (FPA)



FP: Calculation of Function Points /******/

- **Application boundary**: Defines what is “inside” the application vs. “external”; all functions are identified relative to this boundary.
- **ILF (Internal Logical File)**:A logically related data group maintained by the application (persistent logical data store).
- **EIF (External Interface File)**: A logically related data group referenced by the application. A transaction that enters the boundary to add/change/delete ILF data and/or control processing.
- **EO (External Output)**: A transaction that exits the boundary with derived information (processing beyond simple retrieval).
- **EQ (External Inquiry)**: A transaction with an input that triggers immediate retrieval and an output with no significant derivation (query/lookup style).

FP: Calculation of Function Points

- Step 1: Define boundary: Specify the application boundary and external systems.
- Step 2 - Identify functions: Count: EI, EO, EQ, ILF, EIF.
- Step 3 - Classify complexity:Assign each function Low / Average / High using the chosen FP counting rules.
- Step 4 - Apply weights and sum: Multiply each count by its corresponding weight; sum to obtain the weighted totalThis weighted sum is the FP sSoftware Metrics and their use …ttings; if an adjustment step is taught, call this “unadjusted” in that context.
- Step 5 - (Optional) Adjustment: Some methods apply an adjustment factor (method-dependent); if not used, skip.
- Step 6 - Estimation use: Convert FP size to effort/cost via calibrated productivity (e.g., person-hours per FP) derived from historical data.

FP: Calculation of Function Points

TABLE 1: Function point complexity weights.

Measurement parameter	Weighting factor		
	Simple	Average	Complex
Number of user inputs	3	4	6
Number of user outputs	4	5	7
Number of user inquiries	3	4	6
Number of files	7	10	15
Number of external interfaces	5	7	10

FP: Calculation of Function Points

Measurement parameter	Count	Weighting Factor(Average)	Total
Number of user inputs	50	4	=200
Number of user outputs	40	5	=200
Number of user enquiries	35	4	=140
Number of user files	6	10	=60
Number of external interfaces	4	7	=28

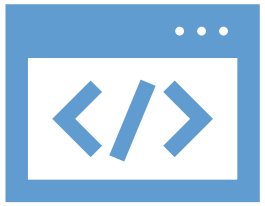
FP: Advantages

They provide a standardized measure of *functional size* (technology-independent)

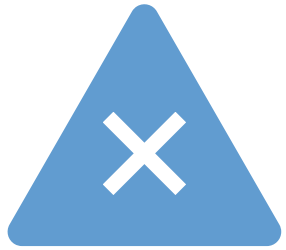
They are based on the user's perspective and the business value of the software application, not on the technology or implementation details

They can be used to estimate software development effort, cost, and schedule, as well as to compare software applications across different domains and platforms

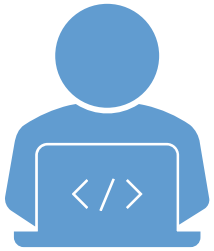
FP: Limitations



They may not capture the full range of software functionality and quality attributes, such as usability, maintainability, and reliability



They may be subject to interpretation and judgment, as the weights and conversion factors may vary across stakeholders and contexts



They may not be suitable for all types of software applications, such as those with a heavy emphasis on user interface or real-time processing.

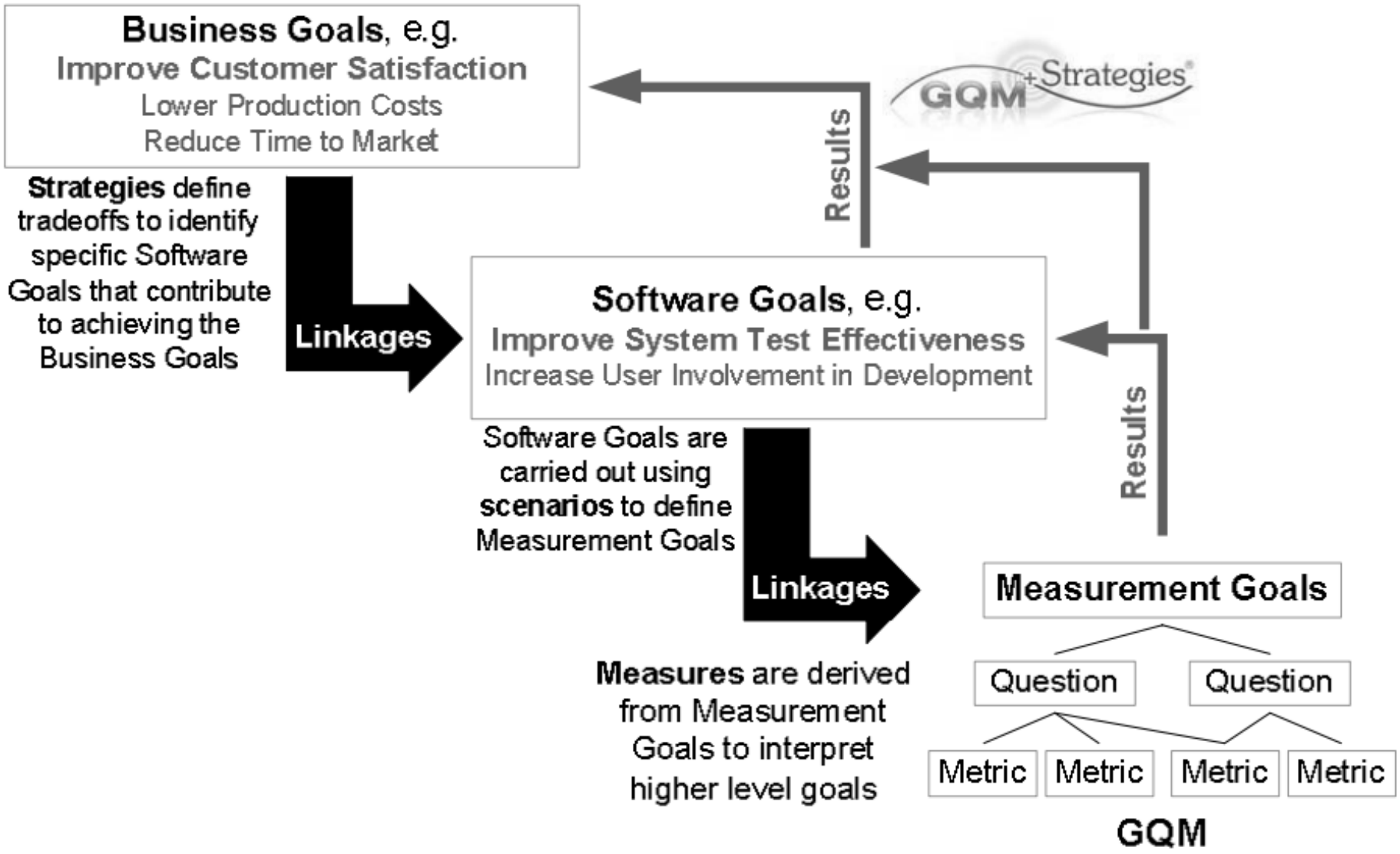
FP: Example 1

- Online Shopping System
  - Internal Logical Files (ILFs): customer accounts, product catalog
  - External Interface Files (EIFs): payment gateway, shipping tracker
  - External Inputs (EIs): add item to the cart, update account details
  - External Outputs (EOs): order confirmation, shipping notification
  - External Inquiries (EQs): search for product availability

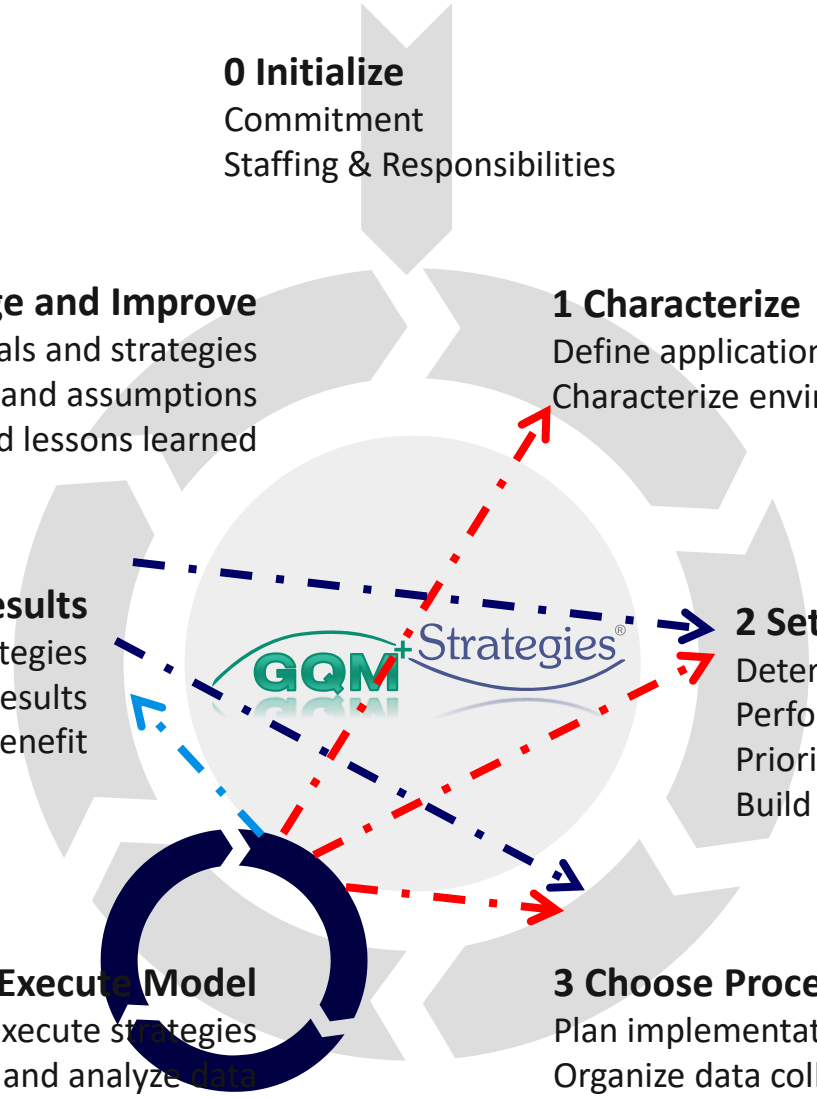
GQM+Strategies

GQM+S: Definition

- GQM+Strategies is a software engineering approach that combines:
  - Goal-Question-Metric (GQM) method to define the goals, questions, and metrics, and
  - Strategies method to identify and select the most appropriate strategies for achieving the goals.
- GQM+S links business goals → strategies → measurement goals to ensure metrics drive organizational decisions.


Business Goals, e.g.
Improve Customer Satisfaction
Lower Production Costs
Reduce Time to Market
GQM+Strategies
Results
Strategies define tradeoffs to identify specific Software Goals that contribute to achieving the Business Goals
Linkages
Software Goals, e.g.
Improve System Test Effectiveness
Increase User Involvement in Development
Results
Software Goals are carried out using scenarios to define Measurement Goals
Linkages
Measurement Goals
Question
Question
Metric
Metric
Metric
Metric
Measures are derived from Measurement Goals to interpret higher level goals
GQM

The GQM + Strategies Reference Process



0 Initialize
Commitment
Staffing & Responsibilities

1 Characterize
Define application scope
Characterize environment/context

2 Set Goals
Determine organizational structures
Perform status quo analysis
Prioritize goals
Build up goals-strategies grid

3 Choose Process
Plan implementation of strategies
Organize data collection and analysis
Define feedback mechanisms

4 Execute Model
4.1 Execute strategies
4.2 Collect and analyze data
4.3 Provide feedback

5 Analyze Results
Analyze data and revise strategies
Review and communicate results
Analyze cost/benefit

6 Package and Improve
Adapt and improve goals and strategies
Revise context and assumptions
Record lessons learned

Modify the target process as needed, if due

Deploy the measurement plan in the target process

Credits. V.R. Basili et al. Aligning Organizations Through Measurement: The GQM+Strategies Approach, Springer, 2014.

GQM+S: TYPICAL USES (GOAL → MEASURES → DECISION)

- **Process improvement**
  - Goal: reduce lead time / rework / defect leakage.
  - Measures: cycle time, WIP, re-opened defects, escaped defects.
  - Decision: change workflow policies; invest in automation; redesign review/testing gates.
- **Project management and tracking**
  - Goal: improve predictability and milestone attainment.
  - Measures: milestone burndown, scope change rate, throughput, risk indicators (e.g., hotspot churn).
  - Decision: re-plan scope; reallocate staffing; adjust release train.
- **Decision support (strategic/technical)**
  - Goal: evaluate adoption of technology/methodology.
  - Measures: pilot outcomes (defect density, performance, cost), learning curve, operational overhead.
  - Decision: adopt/rollback; define rollout constraints and guardrails.

Strategies

- The strategy describes what organizational-technical action is taken.
- The target specifies how success will be judged.
- The metrics operationalize the evaluation.
- The question ensures alignment between goal and evidence.

GQM+S: Examples (1/2)

Goal: Improve code quality	Metric: Code coverage	Strategy: +10% next release by introducing unit tests for critical modules + enforce a coverage gate.	Value: Current code coverage is 70%, target code coverage is 80%
Goal: Improve software testing efficiency	Metric: Test execution time (with defect detection rate held constant)	Strategy: reduce test execution by 5% by test automation optimization via parallel test execution.	Value:
Goal: Enhance user experience	Metric: User satisfaction rating	Strategy:	Value

GQM+S: Examples (2/2)

Goal: Improve code quality	Metric: Code coverage	Strategy: +10% next release by introducing unit tests for critical modules + enforce a coverage gate.	Value: Current code coverage is 70%, target code coverage is 80%
Goal: Improve software development process	Metric:	Strategy:	Value:
Goal: Improve development team productivity	Metric:	Strategy:	Value:

References

- ISO standard:
  - https://www.iso.org/standard/35733.html

- OO Metrics
  - Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object-oriented design. IEEE Transactions on software engineering, 20(6), 476-493.
  - Li, Y., & Henry, S. (1993). Software metrics and software quality assurance. Journal of systems and software, 22(3), 157-165.

- FP
  - Jeffery DR, Low GC, Barnes M. A comparison of function point counting techniques. IEEE Transactions on Software Engineering. 1993 May;19(5):529-32.
  - Garmus D, Herron D. Function point analysis: measurement practices for successful software projects. Addison-Wesley Longman Publishing Co., Inc.; 2001 Jan 1.

- GQM+Strategies
  - Basili, Victor., Münch, Jürgen., Heidrich, Jens., Rombach, Dieter., Seaman, Carolyn., Trendowicz, Adam., Kowalczyk, Martin. Aligning Organizations Through Measurement: The GQM+Strategies Approach. Germany: Springer International Publishing, 2014.